GestureCanvas AI — Detailed Project Report

Report date: 14 October 2026
Project: GestureCanvas AI — browser-based hand-tracking drawing studioSubmitted by : Anish Sharma (70472400391)
Repository: anish3sharma-working/gesturecanvas-ai
Production app: gesturecanvas-ai.vercel.app
Reviewed revision: `7a40ffe` — `feat: pause drawing when hand is too close`

1. Executive summary

GestureCanvas AI is a local-first web application that turns live webcam imagery into an interactive drawing input. When the user explicitly starts the camera, the browser supplies a video stream; a locally executed MediaPipe Hand Landmarker model estimates hand landmarks in successive frames. The application interprets the landmark geometry, stabilizes the index-finger position, maps it from camera-image space into canvas space, and uses it to create editable artwork.

The central technical idea is the separation of **video used as an input signal** from **artwork as retained output**. The live camera preview, hand-landmark overlay, and editable artwork are separate visual layers. The webcam feed is not recorded or uploaded and does not become part of the artwork or PNG export. A simulated hand and mouse/touch input provide camera-free alternatives.

The project is an example of **real-time browser computer vision and interactive graphics**, not a full video-editing or general image-manipulation suite. Its image/video processing scope is specifically live-frame acquisition, learned hand detection and landmark estimation, geometric gesture/proximity analysis, coordinate transformation, visual overlay/compositing, and artwork-only raster export.

2. Project goals and user experience

The project explores touch-free interaction for sketching, demonstrations, and introductory computer-vision learning. Its goals are to:

- Request camera access only after a user action and request video without audio.
- Process camera frames in the browser with a pretrained hand-tracking model.
- Translate hand movement and selected gestures into cursor and drawing behavior.
- Reduce cursor jitter while keeping movement responsive.

- Keep the camera feed separate from artwork and local persistence.
- Offer opt-in pinch drawing, pause gestures, and a close-hand safety pause.
- Remain useful without a camera through the simulated demo and direct pointer drawing.

A typical camera interaction is: **Start camera → grant browser permission → load the local model/runtime → analyze video frames → detect hand landmarks → classify/stabilize/map the hand position → update the drawing layer**. Stopping the camera releases the stream tracks and model resources. The demo path supplies synthetic landmark coordinates and is visibly labeled as simulated; it demonstrates the interface/drawing integration, not real-world model accuracy.

3. Image and live-video processing pipeline

3.1 Frame acquisition and video input

The app obtains a `MediaStream` with `navigator.mediaDevices.getUserMedia` only when the user selects **Start camera**. The requested video dimensions are idealized at 1280 × 720, with a user-facing camera and no audio track. The stream is attached to a muted, inline HTML `<video>` element.

The video is processed as a sequence of frames rather than as a prerecorded movie file. An animation-frame loop checks that the video has a new, ready frame and applies a short interval gate (approximately 28 ms) before inference. This avoids repeatedly analyzing the same frame and constrains the processing cadence. Actual frame rate depends on camera, browser, model, and hardware; no fixed real-time FPS guarantee is made.

3.2 Local learned hand detection and landmark estimation

The app uses the `@mediapipe/tasks-vision` browser package and its Hand Landmarker task model. The WebAssembly vision runtime and model are loaded in the browser; the model is configured for video mode and up to two hands. Detection, presence, and tracking confidence thresholds are each configured to 0.55.

For each detected hand, the model provides a set of **21 normalized hand landmarks**, along with category metadata such as handedness and a confidence score when available. These keypoints correspond to the wrist and finger joints/tips. The app uses a subset for cursor control and geometric reasoning; it can draw a 21-point hand skeleton and landmark dots in the tracking overlay. The index fingertip (landmark 8) is used as the pointing cursor; the thumb tip (landmark 4) participates in pinch detection.

This is pretrained-model inference, not a custom model trained by this project. The app does not perform face detection, facial recognition, identity matching, or user identification.

3.3 Gesture interpretation using image-space geometry

The application treats landmarks as geometric features and applies hand-built rules:

- **Pinch:** measures the normalized distance between thumb tip 4 and index tip 8, divided by the wrist-to-middle-finger-base distance (landmark 9). Separate engage and release thresholds (0.58 and 0.78) provide hysteresis, reducing rapid toggling near the pinch boundary.
- **Finger extension:** compares the distance from the wrist to a finger tip with wrist-to-joint distance. This heuristic contributes to open-palm, fist, index-finger, and two-finger classifications.
- **Temporal stabilization:** a multi-frame debouncer (five consecutive frames by default) prevents a single noisy classification from changing the stable gesture state.
- **Immediate stopping:** enabled open-palm and fist candidates stop the active stroke immediately rather than waiting for the longer general gesture debounce. Pinch release uses a shorter two-frame debounce to balance responsive stopping with resistance to one-frame tracking flicker.

These are interpretable landmark heuristics. They are not a broad gesture-recognition model, and real-world recognition may vary with camera angle, hand pose, occlusion, lighting, and individual differences.

3.4 Hand-proximity / apparent-size safety gate

The close-hand feature estimates **relative apparent hand size in the image**, not physical distance in centimeters. It compares the palm width (index MCP landmark 5 to pinky MCP landmark 17) and palm length (wrist 0 to middle MCP 9), after correcting horizontal coordinates for a 16:9 camera-frame aspect ratio. The larger measurement is treated as the palm-scale estimate.

When the measurement reaches the user-adjustable limit, gesture drawing is paused and the current stroke is finalized. The setting is enabled by default; the interface provides a 25%–75% scale limit with a default of 42%. A four-percentage-point release hysteresis means the hand must move sufficiently back from the activation boundary before drawing can resume, helping avoid repeated pause/resume chatter. The guard fails safe when the landmark set is incomplete, and resets when no hand is detected or the guard is disabled.

This feature is a camera-frame proximity **proxy**. A hand may look large because it is close, but camera zoom, framing, orientation, and perspective also affect apparent size. It should not be interpreted as calibrated depth sensing.

3.5 Cursor filtering and camera-to-canvas transformation

The index-tip landmark is temporally stabilized using an adaptive exponential update. The stabilizer estimates movement speed from displacement divided by elapsed time. Slower movement receives stronger smoothing to suppress jitter; faster movement increases the response factor to reduce lag. Users can select Off, Low, Medium, or High stabilization.

After smoothing, the normalized landmark is mapped from camera-image coordinates into the 1280 × 720 artwork coordinate system. The mapping uses a **cover/crop transform**: a scale large enough to fill the destination is selected, excess dimensions are center-cropped, horizontal position is mirrored to match the selfie-preview interaction, and coordinates are clamped to canvas bounds. This keeps cursor placement aligned with a mirrored, cover-cropped display rather than simply multiplying normalized x/y by canvas dimensions.

The display canvases are resized for their CSS layout and device-pixel ratio, then transformed to the logical artwork dimensions. This supports sharp canvas rendering across display densities while keeping artwork coordinates independent of the screen's pixel dimensions.

3.6 Visual layers and compositing

The workspace layers are intentionally separated:

1. **Video layer:** live mirrored webcam preview, displayed only when a stream is active; it can also be selected as a composition backdrop.
2. **Artwork layer:** Canvas 2D output for persistent editable artwork and the current in-progress preview.
3. **Tracking overlay:** hand skeleton/landmark visualization, cursor indicator, and selection bounds.

The skeleton overlay draws connections between the 21 landmarks and marks the index fingertip distinctly. This is a visualization of model output; it is not itself stored in the artwork. The user can turn landmark display off. The optional camera-feed background is preview-only and excluded from saving and export.

3.7 Drawing primitives and raster operations

The drawing system retains artwork as typed items (paths, shapes, text) rather than as screenshots of the workspace. Canvas 2D renders those items. Pen/marker paths use quadratic interpolation between input points to produce smoother strokes. Lines, rectangles, ellipses, arrows, and text have separate geometry/render paths. Brush size, color, and opacity are applied per item; markers use a wider, lower-opacity rendering style.

Erasing uses Canvas compositing with the `destination-out` operation against artwork. It is a deliberate user-selected tool, not a hand gesture. Optional symmetry mirrors non-text

drawing items around the canvas center. Grid/rule-of-thirds guides are drawn separately from the retained artwork.

3.8 PNG image export

PNG export creates a fresh off-screen HTML canvas at the artwork's logical dimensions, renders drawing items to it, and encodes it with `toBlob` as `image/png`. The export path uses artwork items, not the visible workspace screenshot. It therefore excludes the webcam feed, landmarks, cursor, guides, application controls, and other interface layers. Transparency is supported; camera imagery is never baked into the downloaded image.

4. Functional scope beyond the vision pipeline

The studio includes pen, marker, eraser, line, rectangle, ellipse, arrow, text, selection, object movement, color/size/opacity settings, undo/redo, backgrounds, guides, symmetry, and artwork management. Named artworks are stored in browser-local IndexedDB with a localStorage fallback. This is per-browser storage, not a cloud gallery or account-based synchronization service.

Mouse/touch pointer interaction works independently of camera tracking. The camera-free demo sends synthetic landmarks through the same interaction path, and the interface labels this clearly to distinguish simulation from actual inference.

5. Architecture and technology

- **Frontend:** React 19, TypeScript, Vite.
- **Vision:** MediaPipe Tasks Vision JavaScript package, browser WebAssembly runtime, pretrained Hand Landmarker task model.
- **Graphics:** HTML video and Canvas 2D with multiple visual layers.
- **Local data:** IndexedDB, with localStorage fallback.
- **Validation:** Vitest, React Testing Library, and jsdom.
- **Hosting:** Vercel static deployment. `vercel.json` downloads the model during its build, builds the Vite application, and serves the `dist` output with an SPA route fallback.

There is no application backend or paid inference service in the runtime path. The deployed web application still needs the user's browser camera permission to use live-camera tracking.

6. Privacy and resource lifecycle

Camera access is opt-in. The app requests no audio and does not record or upload camera frames. Video analysis is performed in the user's browser using the local runtime/model assets. Saved art remains in that browser's storage; PNG export is a local download. The camera preview can be displayed for composition, but is not included in the saved artwork or export.

On camera stop or page exit, the app cancels the frame loop, stops media tracks, detaches the video stream, and closes the hand-landmarker resource. Gestures are limited to drawing/pause behavior; they cannot erase artwork or switch tools. Destructive whole-canvas clearing requires confirmation.

7. Testing and evidence

At revision `7a40ffe`, automated validation reported **65 passing tests across 9 test files**, with TypeScript typecheck and production build passing. Coverage includes gesture geometry and thresholds, immediate pause behavior, proximity hysteresis, controls, smoothing, coordinate transforms, drawing and export behavior, selection/history, storage validation, and mocked camera/demo lifecycle behavior.

These checks establish algorithmic/component behavior and build integrity. They do **not** establish recognition accuracy or latency with a physical webcam. The live production page was confirmed to render, but a physical camera trial and end-to-end real MediaPipe inference on varied user hardware remain outstanding. The simulated demo exercises UI/drawing flow only; it is not evidence of model detection quality.

8. Limitations and future work

1. **Apparent distance is not metric depth.** A calibrated depth camera, stereo setup, or validated monocular-depth method would be required for reliable distance in physical units.
2. **Rule-based gesture robustness is unproven across users.** Conduct hands-on trials across cameras, lighting, left/right hands, rotations, skin/background contrast, occlusion, and working distances; tune thresholds based on measured false activation and stop latency.
3. **Main-thread inference can affect responsiveness.** A worker/off-main-thread inference path and measured frame-time telemetry could improve performance on slower devices.
4. **Add confidence-aware behavior.** The app receives model scores but does not yet expose a rich confidence visualization or fully base all gesture decisions on confidence.

5. **Expand accessibility and browser QA.** Test keyboard-only interaction, screen-reader descriptions, reduced-motion preferences, touch devices, camera permission variations, storage quota/private mode, and multiple browser engines.
6. **Strengthen model delivery verification.** Record and validate model version/checksum in deployment and download workflows.
7. **Clarify product positioning.** If the goal expands to image editing or video editing, add explicit file import, image transforms/filters, timeline or clip operations, rendering/encoding, and export workflows. Those capabilities are outside the current implementation.

9. Conclusion

GestureCanvas AI demonstrates a complete interaction chain from live image acquisition through learned visual landmark inference and geometric gesture interpretation to stabilized coordinate mapping and independent Canvas 2D artwork. Its most important design choice is data separation: the video is an input/preview layer, while strokes are retained as editable drawing objects and exported without camera or UI pixels. It therefore provides a strong educational example of browser-based computer vision combined with interactive graphics, while leaving physical-camera robustness and true metric distance detection as explicit areas for future validation and development.